Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 7

Disciplina: Tehnici de programare

Tema: "Exercițiu de pregătire: Trusa de unelte HTTP"

A efectuat : st. gr. SI–262
Inculeț Andrian

A verificat : asistent universitar
Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

Partea 1: Parametri în URL (10 puncte, ~7 min)

Poți trimite valori unui server prin adresă dându-i lui `requests.get` un dicționar în `params=`, în loc să lipești tu textul adresei.

```
r = requests.get("https://httpbin.org/get", params={"q": "python", "page":
"2"})
print(r.url)             # https://httpbin.org/get?q=python&page=2
print(r.json()["args"])  # {'q': 'python', 'page': '2'}
```

Completează cele două funcții:

1. `search(params)` trimite o cerere GET la `/get`, cu dicționarul ca parametri în URL, și returnează `args`, adică parametrii primiți de server (6 puncte).
2. `search_url(params)` returnează adresa completă construită de `requests`, cu tot cu parametri (4 puncte).

Indiciu: încearcă și cu o valoare care conține spații și `&`, de exemplu `{"q": "hello world & co"}`. Dacă lipești adresa de mână, se strică; cu `params=` merge.

```
# Partea 1: parametri în URL
def search(params):
    """GET pe /get cu parametri în URL; returnează parametrii primiți de server."""
    r = requests.get(f"{BASE}/get", params=params, timeout=5)
    return r.json()["args"]

def search_url(params):
    """Returnează adresa completă construită de requests, cu tot cu parametri."""
    r = requests.get(f"{BASE}/get", params=params, timeout=5)
    return r.url
```

Partea 2: Anteturi (10 puncte, ~7 min)

Anteturile îi spun serverului cine ești și ce accepți, iar tu le setezi cu un dicționar în `headers=`.

```
r = requests.get("https://httpbin.org/headers", headers={"User-Agent":
"MyBot/1.0"})
print(r.json()["headers"]["User-Agent"])   # MyBot/1.0
```

Completează cele două funcții:

1. `fetch_with_agent(agent)` trimite o cerere GET la `/headers` cu antetul `User-Agent` setat la `agent` și returnează User-Agent-ul văzut de server (6 puncte).
2. `default_agent()` trimite aceeași cerere fără să setezi vreun antet și returnează User-Agent-ul văzut de server (4 puncte).

Indiciu: `default_agent()` arată de ce unele site-uri blochează scripturile: implicit, `requests` se prezintă ca program Python.

```
# Partea 2: anteturi
def fetch_with_agent(agent):
    """GET pe /headers cu User-Agent=agent; returnează User-Agent-ul văzut de
server."""
    r = requests.get(f"{BASE}/headers", headers={"User-Agent": agent}, timeout=5)
    return r.json()["headers"]["User-Agent"]

def default_agent():
    """Returnează User-Agent-ul trimis de requests când nu setezi nimic."""
    r = requests.get(f"{BASE}/headers", timeout=5)
    return r.json()["headers"]["User-Agent"]
```

Partea 3: Trimiterea datelor (15 puncte, ~12 min)

O cerere POST duce datele în corpul ei, într-unul din două formate frecvente: JSON, cu `json=`, sau câmpuri de formular (ca într-un formular HTML), cu `data=`.

```
r = requests.post("https://httpbin.org/post", json={"name": "Ana"})
print(r.json()["json"])   # {'name': 'Ana'}

r = requests.post("https://httpbin.org/post", data={"name": "Ana"})
print(r.json()["form"])   # {'name': 'Ana'}
```

Serverul răspunde cu o descriere a ceea ce a primit, într-o cheie care poartă numele formatului.

Completează cele două funcții:

1. `post_json(data)` trimite `data` ca JSON la `/post` și returnează partea `json` din răspuns (8 puncte).
2. `post_form(data)` trimite `data` ca câmpuri de formular la `/post` și returnează partea `form` din răspuns (7 puncte).

Indiciu: dacă folosești argumentul greșit, cheia potrivită din răspuns vine goală. Așa deosebești cel mai repede `json=` de `data=`.

```
# Partea 3: trimiterea datelor
def post_json(data):
    """POST cu datele ca JSON pe /post; returnează JSON-ul primit de server."""
    r = requests.post(f"{BASE}/post", json=data, timeout=5)
    return r.json()["json"]

def post_form(data):
    """POST cu datele ca formular pe /post; returnează formularul primit de server."""
    r = requests.post(f"{BASE}/post", data=data, timeout=5)
    return r.json()["form"]
```

Partea 4: Coduri de stare (10 puncte, ~8 min)

Un cod de stare îți spune dacă o cerere a reușit: 2xx înseamnă succes, 4xx înseamnă că ai greșit tu, iar 5xx înseamnă că a greșit serverul. Adresa `/status/<code>` face ca httpbin să răspundă cu orice cod ceri.

Implicit, `requests` nu dă eroare la un 404 sau la un 500. Metoda `r.raise_for_status()` o face: ridică `requests.HTTPError` pentru orice 4xx sau 5xx, iar tu o poți prinde cu `try/except`.

```
r = requests.get("https://httpbin.org/status/404")
print(r.status_code)      # 404
try:
    r.raise_for_status()
except requests.HTTPError as e:
    print("a eșuat:", e)
```

Completează cele două funcții:

1. `describe_status(code)` cere `/status/<code>` și returnează `"succes"` pentru 2xx, `"eroare client"` pentru 4xx și `"eroare server"` pentru 5xx (6 puncte).

2. `is_ok(code)` cere `/status/<code>` și returnează `True` dacă cererea a reușit și `False` dacă nu. Folosește `raise_for_status()` într-un `try/except` (4 puncte).

Indiciu: niciuna dintre funcții nu are voie să se oprească cu eroare la un 404 sau la un 500.

```
# Partea 4: coduri de stare
def describe_status(code):
    """GET pe /status/<code>; returnează 'succes', 'eroare client' sau 'eroare
server'."""
    r = requests.get(f"{BASE}/status/{code}", timeout=5)

    if 200<= r.status_code <= 299:
        return "succes"
    if 400 <= r.status_code <= 499:
        return "eroare client"
    if 500 <= r.status_code <= 599:
        return "eroare server"
    return "altceva"

def is_ok(code):
    """GET pe /status/<code>; returnează True dacă cererea a reuşit, altfel False."""
    r = requests.get(f"{BASE}/status/{code}", timeout=5)

    try:
```

```
        r.raise_for_status()
        return True

    except requests.HTTPError:
        return False
```

Partea 5: Redirecționări (10 puncte, ~10 min)

O redirecționare înseamnă că serverul îți spune "mergi la altă adresă"; `requests` o urmează automat și ține minte fiecare pas în `r.history`. Adresa `/redirect/<n>` te redirecționează de `n` ori, înainte să ajungi la `/get`.

```
r = requests.get("https://httpbin.org/redirect/3")
print(len(r.history))   # 3 (un răspuns pentru fiecare redirecționare)
print(r.url)            # https://httpbin.org/get

r = requests.get("https://httpbin.org/redirect/3", allow_redirects=False)
print(r.status_code)            # 302
print(r.headers["Location"])    # unde voia serverul să te trimită
```

Completează cele trei funcții:

1. `count_redirects(n)` cere `/redirect/<n>` și returnează câte redirecționări au fost urmate (5 puncte).
2. `final_url(n)` returnează adresa la care a ajuns cererea (2 puncte).
3. `first_hop(n)` cere `/redirect/<n>` fără să urmeze redirecționările și returnează antetul `Location` al primului răspuns (3 puncte).

Indiciu: `first_hop` trebuie să se oprească la primul răspuns, deci are nevoie de `allow_redirects=False`.

```
# Partea 5: redirecționări
def count_redirects(n):
    """GET pe /redirect/<n>; returnează câte redirecționări au fost urmate."""
    r = requests.get(f"{BASE}/redirect/{n}", timeout=5)

    return len(r.history)

def final_url(n):
    """GET pe /redirect/<n>; returnează adresa la care ai ajuns."""
    r = requests.get(f"{BASE}/redirect/{n}", timeout=5)
    return r.url

def first_hop(n):
    """GET pe /redirect/<n> FĂRĂ a urma redirecționarea; returnează antetul
Location."""
```

```
    r = requests.get(f"{BASE}/redirect/{n}", allow_redirects=False, timeout=5)
    return r.headers.get("Location")
```

Partea 6: Sesiuni și cookie-uri (10 puncte, ~8 min)

Un cookie este o mică valoare pe care serverul îi cere clientului să o țină minte și să o trimită înapoi, iar `requests.Session()` este obiectul care ține minte cookie-urile între cereri. Fiecare apel simplu `requests.get` pornește de la zero și uită tot.

Adresa `/cookies/set?name=value` setează un cookie, iar `/cookies` arată cookie-urile primite de server.

```
s = requests.Session()
s.get("https://httpbin.org/cookies/set?theme=dark")
r = s.get("https://httpbin.org/cookies")
print(r.json()["cookies"])   # {'theme': 'dark'}
```

Completează cele două funcții:

1. `cookie_demo(name, value)` folosește o singură `Session` ca să seteze cookie-ul, apoi returnează cookie-urile pe care le vede serverul (7 puncte).
2. `no_session_demo(name, value)` face același lucru cu două apeluri simple `requests.get` și returnează cookie-urile pe care le vede serverul (3 puncte).

Indiciu: a doua funcție trebuie să "eșueze", adică să returneze un dicționar gol. Așa vezi la ce folosește o sesiune.

```
# Partea 6: sesiuni Şi cookie-uri
def cookie_demo(name, value):
    """Setează un cookie, apoi citeŞte cookie-urile înapoi folosind O SINGURĂ
sesiune."""
    s = requests.Session()
    s.get(f"{BASE}/cookies/set?{name}={value}", timeout=5)
    r = s.get(f"{BASE}/cookies")
    return r.json()["cookies"]

def no_session_demo(name, value):
    """Fă acelaŞi lucru cu apeluri simple requests.get; returnează cookie- urile
văzute de server."""
    requests.get(f"{BASE}/cookies/set?{name}={value}", timeout=5)
    r = requests.get(f"{BASE}/cookies", timeout=5)
    return r.json()["cookies"]
```

Partea 7: Raportul final (25 puncte, ~25 min)

Combină tot ce ai învățat într-o singură funcție, `report(path)`, care descrie pagina de la `BASE + path` și nu se oprește niciodată cu eroare, apoi folosește-o într-o buclă.

7a. Funcția report (15 puncte). Ea trebuie să returneze un dicționar ca acesta:

```
{
    "status": 200,
    "final_url": "https://httpbin.org/get",
    "redirects": 0,
    "seconds": 0.31,
    "content_type": "application/json",
}
```

Cerințe:

- Folosește un timeout de 3 secunde.
- `final_url` este adresa la care a ajuns cererea, iar `redirects` este numărul de redirecționări urmate.
- `seconds` este durata cererii, ca număr (vezi `r.elapsed`).
- Dacă cererea eșuează (timeout, fără conexiune), returnează `{"status": None, "error": "<mesaj scurt>"}` în loc să te oprești cu eroare.

Puncte: cheile dicționarului (3), valorile pentru o pagină obișnuită (3), redirecționările (3), `seconds` (2), timeout-ul (2), serverul inaccesibil (2).

```
# Partea 7: raportul final
def report(path):
    """Returnează un dicționar care descrie pagina de la BASE + path."""

    try:
        r = requests.get(BASE + path, timeout=3)
        raspuns = {
            "status": r.status_code,
            "final_url": r.url,
            "redirects": len(r.history),
            "seconds": r.elapsed.total_seconds(),
            "content_type": r.headers.get("Content-Type"),
        }

    except requests.exceptions.RequestException as e:
        raspuns = {
            "status": None,
            "error": str(e)
        }
```

```
    return raspuns
```

7b. Bucla din main (10 puncte). În `main()`:

1. Apelează `report` pentru fiecare adresă din `PATHS` și păstrează rezultatele într-o listă (adaugă în fiecare dicționar și adresa, ca să o poți afișa).
2. Afișează doar paginile cu status `200`, sortate după `seconds`, cea mai lentă prima (6 puncte).
3. La final, afișează câte pagini au reușit și câte au eșuat (4 puncte).

```
    rezultate = []
    succes = 0
    esec = 0

    for path in PATHS:
        res = report(path)
        res["path"] = path
        rezultate.append(res)

        if res.get("status") == 200:
            succes += 1
        else:
            esec += 1

    pagini_succ = [rez for rez in rezultate if rez.get("status") == 200]
    pagini_succ = sorted(pagini_succ, key=lambda x: x["seconds"], reverse=True)

    for pagina in pagini_succ:
        print(f"{pagina['path']} -> {pagina['status']}, {pagina['seconds']:.2f}
secunde")

    print(f"Reusite {succes} | Esuate {esec}")
```

Partea 8 (Bonus): Autentificare și module (10 puncte, ~10 min)

Unele pagini cer un nume de utilizator și o parolă. Cu autentificarea HTTP Basic le trimiți ca un tuplu în `auth=`, iar serverul răspunde `401` dacă sunt greșite.

Pagina `/basic-auth/admin/secret` acceptă doar utilizatorul `admin` cu parola `secret`.

```
r = requests.get("https://httpbin.org/basic-auth/admin/secret", auth=
("admin", "secret"))
print(r.status_code)   # 200
```

8a. check_login (6 puncte). Completează `check_login(user, password)`. Ea cere `/basic-auth/admin/secret` cu datele primite și returnează `True` dacă serverul le acceptă, altfel `False`. Nu are voie să se oprească cu eroare când parola e greșită.

```
      # Bonus: autentificare
      def check_login(user, password):
          """Returnează True dacă serverul acceptă datele de autentificare, altfel
False."""
          r = requests.get(f"{BASE}/basic-auth/admin/secret", auth=(user, password))
          return r.status_code == 200
```

Cod pentru httptools.py

```
import requests


BASE = "https://httpbin.org"
PATHS = ["/get", "/redirect/2", "/delay/1", "/status/404", "/delay/5"]
# Partea 1: parametri în URL
def search(params):
    """GET pe /get cu parametri în URL; returnează parametrii primiți de server."""
    r = requests.get(f"{BASE}/get", params=params, timeout=5)
    return r.json()["args"]

def search_url(params):
    """Returnează adresa completă construită de requests, cu tot cu parametri."""
    r = requests.get(f"{BASE}/get", params=params, timeout=5)
    return r.url

# Partea 2: anteturi
def fetch_with_agent(agent):
    """GET pe /headers cu User-Agent=agent; returnează User-Agent-ul văzut de
server."""
    r = requests.get(f"{BASE}/headers", headers={"User-Agent": agent}, timeout=5)
    return r.json()["headers"]["User-Agent"]

def default_agent():
    """Returnează User-Agent-ul trimis de requests când nu setezi nimic."""
    r = requests.get(f"{BASE}/headers", timeout=5)
    return r.json()["headers"]["User-Agent"]

# Partea 3: trimiterea datelor
def post_json(data):
    """POST cu datele ca JSON pe /post; returnează JSON-ul primit de server."""
    r = requests.post(f"{BASE}/post", json=data, timeout=5)
    return r.json()["json"]

def post_form(data):
    """POST cu datele ca formular pe /post; returnează formularul primit de server."""
    r = requests.post(f"{BASE}/post", data=data, timeout=5)
    return r.json()["form"]

# Partea 4: coduri de stare
def describe_status(code):
    """GET pe /status/<code>; returnează 'succes', 'eroare client' sau 'eroare
server'."""
    r = requests.get(f"{BASE}/status/{code}", timeout=5)

    if 200<= r.status_code <= 299:
        return "succes"
    if 400 <= r.status_code <= 499:
```

```
        return "eroare client"
    if 500 <= r.status_code <= 599:
        return "eroare server"
    return "altceva"

def is_ok(code):
    """GET pe /status/<code>; returnează True dacă cererea a reușit, altfel False."""
    r = requests.get(f"{BASE}/status/{code}", timeout=5)

    try:
        r.raise_for_status()
        return True

    except requests.HTTPError:
        return False



# Partea 5: redirecționări
def count_redirects(n):
    """GET pe /redirect/<n>; returnează câte redirecționări au fost urmate."""
    r = requests.get(f"{BASE}/redirect/{n}", timeout=5)

    return len(r.history)

def final_url(n):
    """GET pe /redirect/<n>; returnează adresa la care ai ajuns."""
    r = requests.get(f"{BASE}/redirect/{n}", timeout=5)
    return r.url

def first_hop(n):
    """GET pe /redirect/<n> FĂRĂ a urma redirecționarea; returnează antetul
Location."""
    r = requests.get(f"{BASE}/redirect/{n}", allow_redirects=False, timeout=5)
    return r.headers.get("Location")

# Partea 6: sesiuni și cookie-uri
def cookie_demo(name, value):
    """Setează un cookie, apoi citește cookie-urile înapoi folosind O SINGURĂ
sesiune."""
    s = requests.Session()
    s.get(f"{BASE}/cookies/set?{name}={value}", timeout=5)
    r = s.get(f"{BASE}/cookies")
    return r.json()["cookies"]

def no_session_demo(name, value):
    """Fă același lucru cu apeluri simple requests.get; returnează cookie- urile
văzute de server."""
    requests.get(f"{BASE}/cookies/set?{name}={value}", timeout=5)
    r = requests.get(f"{BASE}/cookies", timeout=5)
    return r.json()["cookies"]



# Partea 7: raportul final
def report(path):
    """Returnează un dicționar care descrie pagina de la BASE + path."""
```

```
    try:
        r = requests.get(BASE + path, timeout=3)
        raspuns = {
            "status": r.status_code,
            "final_url": r.url,
            "redirects": len(r.history),
            "seconds": r.elapsed.total_seconds(),
            "content_type": r.headers.get("Content-Type"),
        }

    except requests.exceptions.RequestException as e:
        raspuns = {
            "status": None,
            "error": str(e)
        }

    return raspuns


# Bonus: autentificare
def check_login(user, password):
    """Returnează True dacă serverul acceptă datele de autentificare, altfel False."""
    r = requests.get(f"{BASE}/basic-auth/admin/secret", auth=(user, password))
    return r.status_code == 200
```

Cod pentru main.py

```
from httptools import *

def main():
    print(search({"q": "python", "page": "2"}))
    print(search({"q": "hello world & co"}))
    print(search_url({"q": "python", "page":"2"}))
    print(fetch_with_agent("MyBot/1.0"))
    print(default_agent())
    print(post_json({"name": "Ana", "age": 19,"tags": ["a", "b"]}))
    print(post_form({"user": "ana", "city":"Bălți"}))
    print(describe_status(200))
    print(describe_status(404))
    print(describe_status(503))
    print(is_ok(204))
    print(is_ok(500))
    print(count_redirects(3))
    print(final_url(2))
    print(first_hop(3))
    print(cookie_demo("theme", "dark"))
    print(no_session_demo("theme", "dark"))
    print(report("/get"))
    print(report("/redirect/2"))
    print(report("/delay/1"))
    print(report("/delay/5"))
    print(check_login("admin", "secret"))
    print(check_login("admin", "gresit"))

    rezultate = []
    succes = 0
    esec = 0
```

```
    for path in PATHS:
        res = report(path)
        res["path"] = path
        rezultate.append(res)

        if res.get("status") == 200:
            succes += 1
        else:
            esec += 1

    pagini_succ = [rez for rez in rezultate if rez.get("status") == 200]
    pagini_succ = sorted(pagini_succ, key=lambda x: x["seconds"], reverse=True)

    for pagina in pagini_succ:
        print(f"{pagina['path']} -> {pagina['status']}, {pagina['seconds']:.2f}
secunde")

    print(f"Reusite {succes} | Esuate {esec}")




if __name__ == "__main__":
    main()
```

Output la rulare main.py

```
(.venv) xperiencez@lenovo-v15-g4-iru:~/Documents/GitHub/tehnici-de-
programare/Laborator7$ python main.py
{'page': '2', 'q': 'python'}
{'q': 'hello world & co'}
https://httpbin.org/get?q=python&page=2
MyBot/1.0
python-requests/2.34.2
{'age': 19, 'name': 'Ana', 'tags': ['a', 'b']}
{'city': 'Bălți', 'user': 'ana'}
succes
eroare client
eroare server
True
False
3
https://httpbin.org/get
/relative-redirect/2
{'theme': 'dark'}
{}
{'status': 200, 'final_url': 'https://httpbin.org/get', 'redirects': 0, 'seconds':
0.71525, 'content_type': 'application/json'}
{'status': 200, 'final_url': 'https://httpbin.org/get', 'redirects': 2, 'seconds':
0.249265, 'content_type': 'application/json'}
{'status': 200, 'final_url': 'https://httpbin.org/delay/1', 'redirects': 0, 'seconds':
1.839418, 'content_type': 'application/json'}
{'status': None, 'error': "HTTPSConnectionPool(host='httpbin.org', port=443): Read
timed out. (read timeout=3)"}
```

```
True
False
/delay/1 -> 200, 1.70 secunde
/get -> 200, 0.81 secunde
/redirect/2 -> 200, 0.16 secunde
Reusite 3 | Esuate 2
```

CONCLUZII

In laboratorul nr 7 am facut o trusa de functii care vorbesc cu un server web, folosind biblioteca requests si site-ul httpbin.org. Practic, am vazut pas cu pas cum functioneaza HTTP si cum se foloseste in Python.

Am invatat sa trimit parametri in adresa cu params=, iar biblioteca se ocupa singura de caracterele speciale (spatii, &), deci nu mai trebuie sa lipesc adresa de mana. Am setat anteturi cu headers= si am vazut ca requests se prezinta implicit ca python-requests, de aia unele site-uri blocheaza scripturile. Am trimis date cu POST in doua feluri, ca JSON (json=) si ca formular (data=), si am observat ca daca folosesc argumentul gresit, cheia din raspuns vine goala.

Am inteles codurile de stare (2xx inseamna succes, 4xx greseala clientului, 5xx greseala serverului) si cum ajuta raise_for_status(), care transforma erorile in exceptii pe care le pot prinde cu try/except. Am urmarit redirectionarile cu r.history, r.url si allow_redirects=False. Cu Session am vazut ca cookie-urile se pastreaza intre cereri, pe cand cu apeluri simple requests.get se pierd. La final am pus totul in functia report, care da un mic raport despre o pagina (status, adresa finala, redirectionari, durata, tip de continut) si nu crapa niciodata, datorita timeout-ului si prinderii erorilor. La bonus am facut autentificarea Basic si am impartit programul in doua fisiere: httptools.py cu functiile si main.py cu programul principal.

Pe parcurs am gresit si eu cateva lucruri: am dat argumente nepotrivite functiilor final_url si count_redirects in report, am scris user si parola direct in adresa la check_login, am lasat un raise_for_status() fara try/except si am incurcat o lista cu un dictionar la afisare. Le-am reparat pe rand si am invatat ca trebuie sa citesc cerintele cu atentie, sa testez fiecare functie separat si sa gandesc ce se intampla cand ceva merge asa cum nu trebuie.

In final, laboratorul m-a ajutat sa inteleg mai bine cum comunica un client cu un server si ce trebuie facut ca un program sa fie mai sigur: o constanta pentru adresa de baza, un timeout la cereri, prinderea erorilor de retea si codul impartit in module.l